Code Walkthrough Report

Madhu, Dhrumil, Kishor, Rasik

April 4, 2025

1 Process Used

Who: Madhu, Dhrumil, Kishor, and Rasik.

What: We examined the application's critical files to see how each component works and how the various features (Canvas, Modal, Edge handling) are implemented. Specifically, we delved into logic that intersects with React Flow's node and edge management, as well as key backend routes.

When: The walkthrough took place during our scheduled group session. We iterated over each segment of the code to identify important logic paths, focusing on potential code smells and trouble spots.

Where: The code is organized within the `frontend` and `backend` directories. We focused on:

- Frontend: Key React Flow components, e.g. `App.tsx`, `shape-node`, `custom-edge` files.
- Backend: Express routes in `dataRoutes.ts` and `userRoutes.ts`, plus `middleware/auth.ts`.

How:

- Gathered the most critical files (e.g., `App.tsx`, `shape-node` components, `dataRoutes.ts`).
- Discussed each file's structure and flow in a group meeting, with each member explaining a part.
- Noted potential issues and common patterns directly in short code snippets.
- Brainstormed immediate solutions or improvements (documented below).

2 Canvas (Main Diagram)

The canvas is the central area where users place nodes, connect them with edges, and interact with the overall diagram. It leverages the React Flow library to track node/edge positions, handle dragging, and manage user interactions.

2.1 Key Implementation Snippet

A core part of the canvas logic resides in `ReactFlowCanvas` (within `src/App.tsx`). Below is a shortened example:

```
function ReactFlowCanvas({
  theme = 'light',
  snapToGrid = true,
  panOnScroll = true,
  zoomOnDoubleClick = false,
}: ExampleProps) {
  const { screenToFlowPosition, setNodes, getNodes, setEdges } =
    useReactFlow<ShapeNode>();
  const [materialEditor, setMaterialEditor] = useState(false);
  const [isModalOpen, setModalOpen] = useState(false);
  const onSave = useSaveDiagram();

  // ... Some Redux logic omitted ...

  // Handle drop event to add new nodes
  const onDrop: DragEventHandler = (evt) => {
    evt.preventDefault();
    const type = evt.dataTransfer.getData('application/reactflow') as
    ↪  ShapeType;
    const model = JSON.parse(evt.dataTransfer.getData('model'));

    const position = screenToFlowPosition({ x: evt.clientX, y:
    ↪  evt.clientY });
    const newNode: ShapeNode = {
      id: Date.now().toString(),
      type: 'shape',
      position,
      data: { type, color: '#ffffff', model },
      // ...
    };

    setNodes((nodes) => [...nodes, newNode]);
  };

  return (
    <ReactFlow
      nodeTypes={nodeTypes}
      edgeTypes={edgeTypes}
      onDrop={onDrop}
      // ...
    >
      {/* Canvas background, side panels, modal components */}
      <Background />
      {/* ... */}
    </ReactFlow>
```

```
  );
}
```

2.2 Explanation

- `useReactFlow()` exposes methods for converting screen coordinates to flow coordinates, adding nodes, and setting edges.
- The `onDrop` handler places new nodes at the drop location. Data about the node (e.g., `model`, shape type) is passed via `dataTransfer`.
- `ReactFlow` is wrapped around child components like `Background`, `MiniMap`, and `Sidebar` (for dragging).

2.3 Process and Observed Issues

- During our walkthrough, the team tested drag-and-drop by placing nodes at various screen coordinates to confirm correct positioning on the canvas.
- We also inspected whether state updates (e.g., `verified` status) were preventing modifications, like removing nodes or dragging them.

2.4 Solutions to Noted Problems

1. **Data Overwrites:** Previously, we cleared `defaultNodes` and `defaultEdges` too aggressively. We made sure to only reset them when necessary (*e.g.*, switching diagrams).
2. **User Interaction Flow:** When `verified` is `true`, deletion or adding new nodes is disabled. The group agreed on providing an in-app message clarifying why the user cannot modify the diagram.
3. **Performance Considerations:** We found that for large diagrams, frequent re-renders could slow the UI. We introduced minimal checks (like `useCallback` or `useMemo`) for large node sets.

3 Modal (Shape Node Modal)

The application uses a floating modal (technically, a `NodeToolbar` overlay) to allow users to edit node details. When a node is selected, this modal appears and displays different tabs: *Settings*, *Node Vars*, *Info*, and *Specs*.

3.1 Key Implementation

Below is an excerpt from `ShapeNodeModal` in `src/components/modal/index.tsx`. It shows how the modal is rendered and which tabs it offers:

```
function ShapeNodeModal({ model, onModelFieldChange,
↪  onNodeVarFieldChange, ... }){
  const [activeTab, setActiveTab] =
  ↪  useState<"settings"|"nodeVars"|"info"|"specs">("settings");

  return (
    <NodeToolbar className='nodrag' offset={32}>
      <div className='form-container'>
        <ModalHeader activeTab={activeTab} onTabChange={setActiveTab} />
        <div className='tab-content'>
          {activeTab === 'settings' && (
            <SettingsTab model={model}
            ↪  onModelFieldChange={onModelFieldChange} ... />
          )}
          {activeTab === 'nodeVars' && (
            <NodeVarsTab model={model} handleChange={onVarChange} ... />
          )}
          {/* ... */}
        </div>
      </div>
    </NodeToolbar>
  );
}
```

3.2 Explanation

- `useState` keeps track of which tab is currently selected.
- `ModalHeader` provides the tab buttons to switch between views.
- Each tab is represented by a separate component (`SettingsTab`, `NodeVarsTab`, etc.), where the user can edit different aspects of the node.

3.3 Process Used (Local Focus)

- We systematically tested each tab to confirm that updates to fields (like `node_name`, model version picks) correctly propagate to Redux.
- Cross-checked the interplay between the `SettingsTab` and `NodeVarsTab` to see if changes in one area could inadvertently affect data in another.

3.4 Results and Solutions

1. **Validation Gaps:** Some fields in `NodeVarsTab` allowed empty values that led to partial node states. We solved it by requiring numeric inputs for crucial ports.
2. **Premature Dismissal of Modal:** The group decided to auto-save changes as soon as the user navigates away from a tab, reducing the chance of data loss.

4 Time Period Management

Nodes in the diagram can store time periods, each referencing a model version. This allows the system to handle multiple model configurations over different intervals.

4.1 Key Implementation Snippet

The code snippet below (from `shape-node/index.tsx`) shows how time periods are added or removed:

```
function onAddTimePeriod(initialData: { fromTP: number; toTP: number;
↪  label: string }) {
  setNodes((nodes) =>
    nodes.map((node) => {
      if (node.id !== id) return node;
      const newTimePeriod: TimePeriod = {
        ...,
        model_version: clonedModelVersions,
        selected_model_version_id: nodeModel.defaultModelVersionId,
      };
      return {
        ...node,
        data: {
          ...node.data,
          model: {
            ...nodeModel,
            timePeriods: [...(nodeModel.timePeriods || []),
            ↪  newTimePeriod],
          },
        },
      };
    })
  );
}
```

4.2 Explanation

- Each node can hold an array `timePeriods`, where each entry has `fromTP`, `toTP`, `label`, and a local copy of `model_version`.
- `onAddTimePeriod` clones the existing model versions, attaches them to the new time period, and sets the selected version ID.

4.3 Process and Observed Issues

- We intentionally created time periods with invalid ranges (like negative or reversed `fromTP` and `toTP`) to see how the UI responded.

- Learned that partial `model_version` references caused silent console errors.

4.4 Solutions

1. Restricted entry for `fromTP` and `toTP` to enforce `fromTP < toTP`.
2. When `selected_model_version_id` is missing, we default to the node's base model version.

5 Edges, Streams, and Connectivity

A critical aspect of the diagram is connecting nodes together with edges, each edge optionally referencing a `Stream` object from the domain. This allows the user to map stream data (temperature, composition, etc.) onto node-to-node connections.

5.1 Key Implementation Snippet

Below is a concise extract from `src/components/custom-edge/index.tsx`, showing how an edge's `data.stream` is assigned:

```
const CustomEdge: FC<EdgeProps<Edge<{ stream: Stream }>>> = ({ id, label,
↪  ... }) => {
  const { setEdges } = useReactFlow();

  const setLabel = (stream: Stream | undefined) => {
    setEdges((edges) =>
      edges.map((edge) => {
        if (edge.id === id) {
          return {
            ...edge,
            data: { stream: stream || null },
            label: stream ? stream.instance : null,
          };
        }
        return edge;
      })
    );
  };

  return (
    <>
      <BaseEdge id={id} path={edgePath} markerEnd={markerEnd} />
      <EdgeLabelRenderer>
        {selected ? (
          <EdgeLabelSelector
            ...
```

```
            onLabelChange={setLabel}
          />
        ) : (
          <EdgeLabel labelX={...} labelY={...} name={data?.stream?.name}
          ↪  />
        )}
      </EdgeLabelRenderer>
    </>
  );
};
```

5.2 Explanation

- When a user picks a `Stream` in the edge modal, `setLabel` updates the edge's `data.stream` with the chosen stream details.
- If no stream is selected, the data remains `null`. This ensures each edge can optionally carry domain stream data.

5.3 Process and Findings

- We used test data in the `MaterialEditor` to see if edge labels updated properly.
- We intentionally selected "invalid" streams to ensure the fallback logic triggered.

5.4 Solutions

1. Added a check that if `stream.instance` is `undefined`, the label defaults to "(No instance)".
2. Added a required field for `aspen_id` in `EdgeLabelSelector` so that each new stream must have a unique ID.

6 Sidebar (Node Palette)

The sidebar provides a palette of available models that the user can drag onto the canvas. Each model is derived from the domain data fetched at runtime.

6.1 Key Implementation Snippet

Below is a shortened excerpt from `src/components/sidebar/index.tsx`:

```
function Sidebar() {
  const domainData = useSelector((state: RootState) =>
  ↪  state.domain.data);

  if (!domainData) {
    return null;
  }

  return (
    <div className='sidebar'>
      <div className='sidebar-label'>Pallete</div>
      <div className='sidebar-items'>
        {domainData.models.map((model) => (
          <SidebarItem
            key={model.id}
            type='rectangle'
            model={model}
          />
        ))}
      </div>
    </div>
  );
}
```

6.2 Explanation

- The `domainData.models` array is mapped to individual `SidebarItem` components.
- Each `SidebarItem` can be dragged onto the canvas. The drag event includes metadata about the model (`model.id`, etc.).

6.3 Process and Resolution of Issues

- **Process:** We tested dragging multiple models in rapid succession. We also tried dropping them onto an invalid zone in the canvas to ensure error handling worked.
- **Solutions:** The team agreed on grouping or searching in the sidebar if we exceed a certain threshold of models, though that is planned for a future iteration.

7 Backend Routes & Data Handling

The backend code uses `Express` and `Prisma` to handle CRUD operations on both MongoDB (for users, diagrams) and PostgreSQL (for domain data). Key logic is defined in `dataRoutes.ts`.

7.1 Key Implementation Snippet

One important example is saving a diagram and its snapshot. Below is a shortened version of the POST route in `dataRoutes.ts`:

```
router.post('/diagrams', authenticateToken, async (req, res, next) => {
  try {
    const userId = req.user.id;
    const { name, canvas, description, domainId, snapshotData, calcType }
    ↪  = req.body;

    // Create an optional domain snapshot if domainId is provided
    let snapshotId: string | undefined;
    if (domainId && snapshotData) {
      const newSnapshot = await mongoClient.domainSnapshot.create({
        data: { userId, domainId, data: snapshotData }
      });
      snapshotId = newSnapshot.id;
    }

    // Generate 'parameters' object from the canvas using translation()
    const parameters = translation(canvas, name, description, domainId,
    ↪  calcType);

    // Create the diagram
    const newDiagram = await mongoClient.diagram.create({
      data: {
        userId,
        name,
        description,
        canvas,
        parameters,
        snapshotId
      },
    });

    res.status(201).json({ message: 'Diagram created successfully',
    ↪  diagram: newDiagram });
  } catch (error) {
    next(error);
  }
});
```

7.2 Explanation

- `authenticateToken` ensures only logged-in users can call this route.
- If `domainId` and `snapshotData` are provided, a `DomainSnapshot` record is created and its `id` is linked to the diagram.

- The `translation(...)` function converts the front-end's `canvas` data into a specialized structure stored in `parameters`.
- `mongoClient` is used here because diagrams reside in MongoDB (unlike domain info stored in PostgreSQL).

7.3 Process & Issues Found

- We tested passing incomplete `canvas` data to the endpoint, verifying that the route gracefully handled the error.
- Observed potential confusion in referencing domain vs. snapshot data across two databases.

7.4 Solutions

1. We introduced an extra check that ensures `snapshotData` must match the `domainId` in the client. If it mismatches, the server returns an explanatory error.
2. Detailed error handling was added around `translation(...)` so that the user receives clearer messages if the canvas fails to parse.

8 Auth & Routing

Authentication: The application uses JWT-based auth. On login, the back-end generates an `accessToken` and `refreshToken`, storing the refresh token in MongoDB.

8.1 Key Implementation Snippet

Below is an excerpt from `userRoutes.ts` illustrating the `/login` logic:

```
router.post('/login', async (req: Request, res: Response) => {
  const { email, password } = req.body;
  const user = await mongoClient.user.findFirst({ where: { email } });

  if (!user || !compareSync(password, user.password)) {
    res.status(401).json({ message: 'Invalid email or password' });
    return;
  }

  const { accessToken, refreshToken } = generateTokens(user.id);
  await mongoClient.refreshToken.create({
    data: {
      token: refreshToken,
```

```
      userId: user.id,
      expiresAt: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000)
    },
  });
  res.json({ name: user.firstName + ' ' + user.lastName, accessToken,
  ↪  refreshToken });
});
```

8.2 Explanation

- If the user exists and `password` matches, `generateTokens(...)` creates two JWTs: an access token (expires faster) and a refresh token (longer-lived).
- The refresh token is stored in the database, allowing the server to invalidate it if necessary.
- The front-end uses the access token for most requests, and can request a new token with the refresh token when it expires.

8.3 Process and Issues

- We tested concurrency by making multiple near-simultaneous requests with expired `accessToken`.
- Saw confusion if a user attempts to log in with a valid token already present but expired refresh token.

8.4 Solutions

1. Added a `token` route that gracefully provides a new `accessToken` if the `refreshToken` is still valid.
2. Synchronized token renewal so that only one refresh request is made at a time, minimizing race conditions.

9 Conclusion: Highlights

Our main findings, along with how we solved or mitigated them, are detailed below:

1. **Data Flow Between Canvas and Redux:** Some sections appear overly coupled to Redux. For instance, in `ReactFlowWrapper` we see logic that resets arrays `defaultNodes` and `defaultEdges` each time a user loads the diagram:

```
useEffect(() => {
  dispatch(clearDomainData());
  if (diagramId) {
    dispatch(clearCanvasName());
  }
  defaultNodes.splice(0, defaultNodes.length);
  defaultEdges.splice(0, defaultEdges.length);
}, [domainId, diagramId, dispatch]);
```

Issue: While this works, it can accidentally overwrite existing state if not used carefully. We noted a risk of data loss if the arrays are cleared prematurely.

Solution: We added conditional checks to confirm `diagramId` is valid before clearing arrays. We also introduced a separate "unsaved changes" flag to track whether clearing is safe.

2. **Edge Label Updates and Stream Selection:** The `CustomEdge` logic in `custom-edge/index.tsx` updates an edge's *stream data* whenever a user picks a stream from the `EdgeLabelSelector`:

```
const setLabel = (stream: Stream | undefined) => {
  setEdges((edges) => edges.map((edge) => {
    if (edge.id === id) {
      return {
        ...edge,
        data: { stream: stream ? stream : null },
        label: stream ? stream.instance : null,
      };
    }
    return edge;
  }));
};
```

Issue: If `stream.data` is left empty, the system might not fully validate it. This could lead to partial or incorrect stream connections.

Solution: We implemented optional validation to ensure that a minimal set of stream properties is present (e.g. `aspen_id`, `content`). If absent, the edge label reverts to a default label (like `"Unnamed Stream"`).

3. **Time Periods & Model Version Mismatch:** In `shape-node/index.tsx`, we allow attaching different `TimePeriod` objects, each referencing their own `model_version`. We found that users could create a Time Period with no valid `selected_model_version_id`, causing silent errors when reading or updating node variables.

Solution: We added UI checks that force the user to select at least one model version whenever a time period is created. If none is chosen, the code defaults to a "base" model version or displays a warning.

4. **Inconsistent Unit Conversions:** The `convertToGivenUnit` function in `modal/utils.ts` tries to handle multiple conversions via base units, but not all domain data includes valid `base_unit` definitions. This occasionally caused crashes. For instance, if `dimension` is missing or spelled differently in `unitsMap`, the function returns the original value with no warning.

   Solution: We now log a console warning whenever a dimension is unknown. Additionally, we introduced a fallback "no conversion" scenario that prompts the user to fix or reassign the dimension before saving.

Overall, we have a functional system, but these issues highlight the need for more robust validation, clearer data handling, and improved coupling between components and Redux. We solved them mostly through conditional checks, default fallbacks, and warnings to the user, while planning more comprehensive fixes for future work.

10 Next Steps

Next Steps:

1. **Allow a Viewer for Aggregated Time Periods Across the Canvas:** Provide a specialized interface where all time periods from every node are combined or summarized in one view. This would allow users to quickly gauge total simulation spans, see overlapping intervals, and more easily identify missing or conflicting time ranges.

2. **Tighten Up the Views (Per Dr. Mahalec's Requests):** Refine and simplify the node and modal layouts so that required fields and significant warnings are always visible. Incorporate Dr. Mahalec's suggested "good to have" UI features (like tooltips, inline validations) to give clear at-a-glance feedback without crowding the screen.